

Agentic Retrieval-Enrichment for Context-Aware Generative Recommendation

Minkun Zhao
South China University of Technology
Guangzhou, China
csmkzhao@mail.scut.edu.cn

Shaohuai Peng
South China University of Technology
Guangzhou, China
bweried@gmail.com

Zijian Zhang
Johns Hopkins University
Baltimore, USA
zzhan401@jh.edu

Lianghao Xia
Harbin Institute of Technology
Shenzhen, China
xialh@hit.edu.cn

Yong Xu
South China University of Technology
Guangzhou, China
yxu@scut.edu.cn

Liqiang Nie
Harbin Institute of Technology
Shenzhen, China
nieliqiang@gmail.com

Abstract

Generative recommenders such as TIGER and OneRec represent items with semantic identifiers (SIDs) and decode them autoregressively, unifying retrieval, ranking, and generation in a single decoder. In practice, however, the prompt these models receive is little more than a flat interaction history: it omits why past actions matter, how long-term taste differs from short-term intent, what explicit feedback the user gave, how behavior transfers across domains, and how similar users behave. We present AREC, a model-preserving agentic retrieval-enrichment layer for SID-based generative recommendation. AREC treats the recommender as the language core of a personal agent and enriches its input rather than its architecture: it builds four typed memory stores and exposes them through five typed tools. For each request, a planner selects task-relevant tools, an executor resolves their dependencies and assembles a typed evidence graph, and a compiler compresses that graph under a token budget into a compact natural-language context card while preserving SID tokens verbatim. The card is prepended to the prompt, and the recommender is adapted with lightweight LoRA-based supervised fine-tuning so that it learns to read the enriched context. On the public RecIF-Bench, AREC improves performance across recall-oriented, binary, and free-form personalized tasks while leaving capabilities outside its scope intact. These results suggest that typed memory, adaptive tool use, and faithful context compilation can deliver personal-intelligence recommendation around an unmodified generative core.

CCS Concepts

• **Information systems** → **Recommender systems**; *Retrieval models and ranking*; • **Computing methodologies** → **Multi-agent systems**.

Keywords

Generative Recommendation, Agentic Retrieval-Augmented Generation, Personalization

1 Introduction

Personal intelligence is increasingly delegated to agentic systems that observe a user over time and act on their behalf, and recommendation is its canonical instance. A personal recommender must infer more than the next item from a stream of interactions. It must maintain a stable view of the user while remaining sensitive to

recent intent, feedback, and domain shifts. An influential recent line of work, *generative recommendation*, casts this problem as next-token prediction over learned *semantic identifiers* (SIDs) [3]: each item is compressed into a short tuple of codebook tokens, and a decoder is trained to generate the next item’s SID conditioned on the user’s history [2, 11, 20, 21]. The appeal is structural: a single objective subsumes recall and ranking, and the same decoder can additionally produce free-form rationales [5, 8].

Yet the conditioning these models receive remains thin. A typical prompt is a flat concatenation of historical SIDs—a transcript of what the user clicked—with no structure separating long-term taste from this morning’s curiosity, no isolation of items the user actively liked from items merely impressed upon them, no neighborhood signal from similar users, and no transfer evidence across domains. The model is forced to re-discover this structure from scratch on every forward pass. In LLM practice, the standard remedy for impoverished conditioning is retrieval augmentation [6]: fetch relevant context and place it in the prompt. Recent recommender work has adopted retrieval augmentation [16, 18], mostly by retrieving items close to the current query. This helps with local similarity, but it does not expose the user-level evidence needed here: stable preferences, current intent, explicit feedback, cross-domain cues, and neighbor behavior.

We take the view that a generative recommender, much like an LLM-based personal agent, is most useful when given a typed memory and a small catalog of tools that turn its flat history into structured, heterogeneous evidence. We present **AREC** (Agentic Retrieval-Enrichment for Context-aware recommendation), a layer that sits in front of a SID-based recommender without altering its architecture. AREC places three modules before the generator: a *planner* that selects which tools to invoke for a given task, an *executor* that runs the tools against offline stores, resolves typed inter-tool dependencies, and assembles a typed *evidence graph*, and a *compiler* that prunes that graph to a token budget and serializes it into a single compact natural-language *context card*. The card preserves the discrete SID vocabulary character-for-character and is prepended to the prompt; the base recommender is then lightly adapted with LoRA to read it. Figure 1 shows the architecture end to end, together with the four offline memory stores, the five typed tools, a worked evidence graph, and the training/serving invariants the layer enforces.

Our main contributions are as follows.

- We introduce an architecture-agnostic agentic enrichment layer for SID-based generative recommenders. The layer builds four typed offline memory stores and exposes them through five typed tools, so that each request can be enriched with long-term profile, recent intent, labeled behavior, cross-domain transfer, and collaborative evidence without changing the base recommender.
- We design a planner–executor–compiler pipeline that selects task-relevant tools, resolves inter-tool dependencies, assembles a typed evidence graph, and compresses the graph into a budget-bounded natural-language context card. The compiler preserves SID tokens verbatim, so the enriched prompt stays compatible with the original SID vocabulary and tokenizer.
- We adapt the unmodified SID generator with lightweight LoRA-based supervised fine-tuning and evaluate AREC on the public RecIF-Bench, where enrichment improves performance across recall-oriented, binary, and free-form personalized tasks.

We see AREC as a small but concrete instance of a broader agentic personal-intelligence pattern. This pattern is attractive precisely because it decouples *what the model knows about the user* from *how the model is trained*, making personalization improvements composable with ongoing advances in the generative recommender.

2 Related Work

2.1 Generative Recommendation

Generative recommenders replace the retrieve-then-rank pipeline with autoregressive decoding over discrete item codes. TIGER [11] introduced RQ-VAE-based semantic identifiers and showed that a sequence-to-sequence model can decode the next item’s code directly, while a parallel line studies how such codes should be learned and how they couple with language models [4, 15]: LC-Rec [21] integrates collaborative semantics into an LLM’s vocabulary, HSTU [20] scales generative recommendation to industrial sequence lengths, and recent handbooks distill the design space of codebooks, tokenization, and training recipes for SID-based recommenders [5]. The OneRec series develops this paradigm at scale: it unifies retrieval and ranking with a generative decoder and iterative preference alignment [2], extends the recipe in successive technical reports [23], adds in-text reasoning for explanation [8], and most recently releases an open foundation model together with the RecIF-Bench suite we evaluate on [22]. Across these systems the model’s conditioning is predominantly the user’s raw item sequence, optionally interleaved with a textual user portrait. AREC leaves the generator untouched and instead enriches *what* it conditions on with typed, structured evidence; in this sense it is complementary to, rather than competing with, the scaling and alignment advances that drive the foundation models themselves.

2.2 Agentic Retrieval-Augmented Generation

Retrieval-augmented generation [6] pairs a retriever with a generator and is now standard for knowledge-intensive LLM tasks; in recommendation it has been used to bring collaborative or world knowledge into LLM-based recommenders [16, 18]. A parallel thread adapts LLMs into recommenders directly [1, 7], while a more recent

agentic reading frames a capable assistant as a language core augmented with memory and tools, planning over a catalog, resolving dependencies, and sometimes self-correcting [12, 13, 19], and surveys of LLM-powered recommender agents organize such systems around profile, memory, planning, and action modules [9, 17]. Two distinctions matter for our setting. First, AREC performs *heterogeneous, typed* retrieval—long-term profile, recent intent, labeled behavior, cross-domain transfer, and neighbors—rather than the *homogeneous* retrieval of more similar items, because typed evidence is what a personalized recommender most obviously lacks. Second, AREC’s generator emits SID tokens rather than text, so its language-style context card, in the spirit of natural-language user profiles [10, 24], must interleave prose with the model’s discrete item tokens without disturbing them. Unlike open-ended agents, AREC keeps a rule-based planner and compiler with a cold-start fallback, so a malformed plan or compilation never disrupts the recommender at runtime.

3 Methodology

3.1 Problem Setup

Let a user u have an interaction history H_u , and let f_θ be a SID-based generative recommender that decodes a ranked list of item SIDs from a chat-formatted prompt $p(H_u, \tau)$ for task type τ . We keep this generator unchanged and intervene only in its input, since our goal is to improve what the model can condition on without altering the SID vocabulary or the decoding procedure. Following the base model, each item i is encoded as a length-three SID triple wrapped by sentinel tokens,

$$\text{sid}(i) = \langle \text{sid_begin} \rangle \langle s_i^a \rangle \langle s_i^b \rangle \langle s_i^c \rangle \langle \text{sid_end} \rangle, \quad (1)$$

that is, five tokens per item, where (s_i^a, s_i^b, s_i^c) are the hierarchical codebook indices. AREC replaces the original prompt with an enriched prompt

$$\tilde{p}(H_u, \tau) = c(g(H_u, \tau)) \parallel p(H_u, \tau), \quad (2)$$

where g is the planner–executor chain that produces a typed evidence graph G , c is the compiler that serializes G into a context card, and $\parallel$ denotes prepending the card to the original prompt. The base model is then lightly fine-tuned on pairs $(\tilde{p}, y)$, where y is the ground-truth SID answer. This formulation isolates the role of enrichment: the model still predicts the same answer format with the same tokenizer, but it receives a more informative view of the user’s history before decoding. Every SID string inside the card is therefore kept byte-identical to the token sequence already used by f_θ , so Eq. (2) changes only the conditioning context and not the output space.

3.2 Typed Memory Stores

AREC precomputes four stores from the training corpus and keeps them available at inference time. We use offline stores because the evidence needed for personalization is relatively stable, while the downstream request must be served with low latency. The **ProfileStore** records, for each user, the most frequent SIDs, the SIDs carrying positive feedback such as like, follow, and forward, and aggregate engagement statistics that summarize long-term taste. The **LabelBehaviorStore** keeps per-user SID lists sliced by

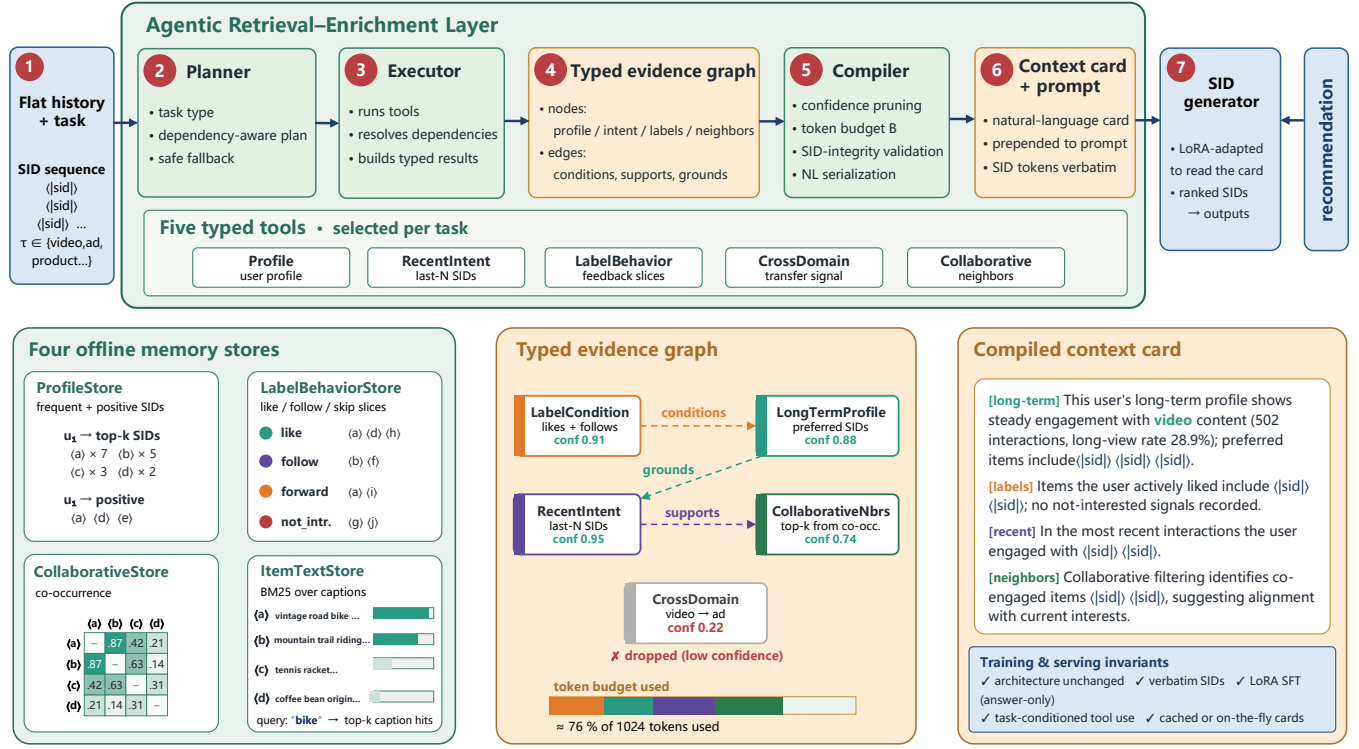


Figure 1: Overview of AREC. A planner-executor-compiler chain turns a flat interaction history into a typed evidence graph and a compact context card, which is prepended to the prompt of an unmodified SID-based generative recommender. Lower panels detail the four memory stores, a worked evidence graph in which a low-confidence node is pruned under the token budget, and a representative context card with byte-identical SID tokens.

interaction type, including longview, like, follow, forward, and not_interested, so that label-conditioned tasks can access the relevant feedback directly instead of forcing the generator to infer it from position alone.

The **CollaborativeStore** is an item-item co-occurrence matrix M built over training sequences with a sliding window and exponential time decay. For items i, j co-occurring at sequence positions t_i, t_j ,

$$M_{ij} = \sum_{(t_i, t_j): |t_i - t_j| \leq w} e^{-\lambda |t_i - t_j|}, \quad (3)$$

where w is the window size and λ is the decay rate. The decay gives more weight to nearby interactions, which are usually more informative for sequential recommendation than distant co-occurrences. The matrix is frequency-filtered and stored sparsely for fast top- k neighbor lookup. The **ItemTextStore** is a BM25 index over item captions. It is used by CrossDomainTool and by the CandidateItemCard construction path when a task requires item-level textual grounding. These stores cover complementary evidence types, including long-term preference, explicit feedback, collaborative regularity, and textual semantics, which are precisely the signals that a flat SID history tends to obscure.

3.3 Typed Tools

Each store is exposed through one or more tools with a shared output schema. A tool returns a typed result $r = (\text{SUMMARY}, \text{SIDS}, \text{TYPE}, \kappa, \hat{n})$, where SUMMARY is a short natural-language description, SIDS contains the relevant SID strings, TYPE specifies the evidence type consumed by the compiler, $\kappa \in [0, 1]$ is a confidence score, and $\hat{n}$ is an estimated token cost. This shared schema is important because the compiler must compare heterogeneous evidence under one token budget rather than handle each store with a separate hand-written rule.

ProfileTool retrieves frequent and positively labeled SIDs from the user's profile. RecentIntentTool maps the last N interactions to SID strings. LabelBehaviorTool returns the latest k SIDs within each feedback slice. CrossDomainTool extracts transfer evidence from active source domains, such as video signals for ad or product tasks. CollaborativeTool queries the sparse co-occurrence matrix M with recent or planner-specified SIDs and returns merged top- k neighbors. The collaborative query SIDs do not have to be fixed before execution. The planner may instead express them as a deferred reference to the recent-intent output, which the executor resolves at runtime. This design keeps individual tools simple while still allowing them to depend on one another when the task requires it. If a user is missing from a store, the corresponding tool builds an inline approximation from the request's own history fields, so the

enrichment layer degrades gracefully rather than hard-failing in cold-start cases.

3.4 Task-Conditioned Planner

The planner maps each request type τ to an ordered tool sequence $\pi(\tau) = (a_1, \dots, a_m)$. Its role is not to produce recommendations, but to decide which evidence should be made visible to the generator before decoding. A single fixed retrieval recipe would be inefficient and often noisy, because different RecIF tasks expose different gaps in the raw history. Short-video recommendation mainly needs profile, recent-intent, and collaborative evidence. Ad and product recommendation additionally need cross-domain evidence, since commercial intent may be implied by earlier content behavior. Label-conditioned recommendation uses profile, label-behavior, recent-intent, and collaborative evidence, because the task is defined by a specific feedback condition.

The planner is also availability-aware. Label-behavior tools are skipped when labeled interactions are absent, and collaborative tools are skipped when the history is too short to form reliable queries. This avoids adding weak evidence merely for completeness. The resulting plan is dependency-aware as well. Collaborative retrieval may depend on the recent-intent output, expressed as a deferred reference such as `$recent_intent.recent_sids[:3]`, which the executor resolves after the upstream tool has produced its result. In this way, the planner remains lightweight while still allowing different task types to receive different evidence mixtures.

3.5 Executor and Typed Evidence Graph

The executor runs $\pi(\tau)$ sequentially, resolves deferred parameters of the form `$tool.field[slice]` against previously computed results, and assembles the outputs into a typed evidence graph $G = (V, E)$. Each node $v \in V$ carries a type drawn from `{LONGTERMPROFILE, RECENTINTENT, LABELCONDITION, CROSSDOMAINSIGNAL, COLLABORATIVENEIGHBORS, CANDIDATEITEMCARD}`, together with its confidence κ_v and token cost estimate $\hat{n}_v$. Each edge $e \in E$ carries a relational type from `{CONDITIONS, SUPPORTS, GROUNDS}`, inferred from a small static rule table. For example, a `LABELCONDITION` node *conditions* the `LONGTERMPROFILE`, while a `COLLABORATIVENEIGHBORS` node *supports* the `RECENTINTENT`, as illustrated in Figure 1.

The graph representation is deliberately more structured than a list of retrieved snippets. Personalization evidence often matters not only because it exists, but also because of how it relates to the current task. A labeled-behavior node constrains the interpretation of the profile, while a collaborative-neighbor node strengthens or weakens the recent-intent signal. Encoding these relations gives the compiler a principled basis for deciding what to keep when the token budget is tight.

3.6 Evidence-Graph Compilation

The compiler turns G into a context card with at most B tokens. It first orders nodes by type-aware priority and confidence, then admits nodes greedily while respecting the budget. Formally, it selects a node subset $S \subseteq V$ according to

$$\max_{S \subseteq V} \sum_{v \in S} \kappa_v \omega_{\text{type}(v)} \quad \text{s.t.} \quad \sum_{v \in S} \hat{n}_v \leq B, \quad (4)$$

where $\omega_{\text{type}(v)}$ is a fixed per-type weight. We solve Eq. (4) greedily in priority order. This choice is sufficient at the small node counts produced by our tools and has a practical advantage: low-value sections are dropped as complete units rather than being cut mid-sentence. The resulting card is written in ordinary prose rather than a heavily tagged format, because the base decoder has already been trained to process natural-language prompts. The card therefore exposes structure without moving the input too far away from the model’s pre-existing prompt distribution.

A central implementation invariant is SID integrity. Whenever a SID appears in the card, the entire sentinel-wrapped sequence is preserved without rewriting, splitting, or paraphrasing. This is necessary because the generator still decodes over the original SID vocabulary. AREC changes the evidence provided to the model, not the representation of the target items.

3.7 Light Supervised Fine-Tuning

The final step teaches the base recommender to use the context card. Although the enriched prompt is deliberately written in a familiar natural-language style, the base model has not been trained to treat such cards as task-relevant evidence. We therefore apply lightweight supervised fine-tuning rather than relying on prompting alone. For each enriched RecIF training sample, we determine the task type, obtain a card from the planner-executor pipeline or from a pre-computed card cache, prepend it to the original prompt as in Eq. (2), and optimize the standard next-token objective on the answer span only,

$$\mathcal{L}(\theta) = - \sum_t \log f_\theta(y_t \mid \tilde{p}(H_u, \tau), y_{<t}), \quad (5)$$

with the prompt span masked out of the loss. Masking the prompt prevents the model from being trained to reproduce the card and focuses on learning to use the card to predict the correct answer. When an enriched prompt exceeds the length budget, we truncate from the middle of the prompt while keeping a short system header and the tokens nearest the answer, so the answer span is always preserved. Samples whose answer alone exceeds the budget are skipped during training.

RecIF SFT samples are reconstructed from the public release split for the supported personalized tasks, including video, ad, product, label-conditioned, and label-prediction settings. We mix them with a small fraction of general-instruction data to preserve the base model’s general generation ability. All samples are drawn from training users only and verified by an explicit user-ID disjointness check against the benchmark split.

4 Experiments

4.1 Experimental Setup

We use OneRec-1.7B [22] as the base SID-based generative recommender and evaluate on the public RecIF-Bench [22], which spans eight personalized tasks grouped into three families. The *recall-style* problems ask the model to generate the SIDs of the next items the user will engage with. The *binary* problems ask for a yes/no judgment on whether the user will engage with a candidate. The *free-form* problems ask for natural-language item descriptions and recommendation rationales. AREC sits in front of the base model:

Table 1: Results on the public RecIF-Bench across eight personalized tasks. Columns are the leaderboard baselines at a comparable scale plus AREC (our enrichment layer on top of ONEREC-1.7B). Baseline numbers are transcribed from the public leaderboard. The best result in each row is in bold. “–” denotes a task a model does not support.

Task	Metric	SASRec	BERT4Rec	GRU4Rec	HSTU	ReaRec	TIGER	OneRec-1.7B	AREC(OneRec-1.7B)
Short Video Rec	Pass@1	0.0045	0.0040	0.0051	0.0043	0.0052	0.0168	0.0496	0.0535
	Pass@32	0.1003	0.0951	0.0993	0.1010	0.1002	0.1061	0.1710	0.2120
	Recall@32	0.0119	0.0113	0.0117	0.0119	0.0120	0.0132	0.0272	0.0341
Ad Rec	Pass@1	0.0044	0.0061	0.0059	0.0076	0.0035	0.0125	0.0169	0.0199
	Pass@32	0.0980	0.1225	0.1102	0.1266	0.1054	0.1769	0.2037	0.2305
	Recall@32	0.0293	0.0381	0.0336	0.0409	0.0327	0.0581	0.0707	0.0826
Product Rec	Pass@1	0.0052	0.0054	0.0047	0.0055	0.0030	0.0120	0.0144	0.0359
	Pass@32	0.0914	0.0936	0.0821	0.0914	0.0907	0.1276	0.1571	0.2680
	Recall@32	0.0175	0.0193	0.0161	0.0178	0.0189	0.0283	0.0360	0.0705
Label-Cond. Rec	Pass@1	0.0026	0.0026	0.0032	0.0026	0.0027	0.0044	0.0064	0.0080
	Pass@32	0.0380	0.0372	0.0393	0.0383	0.0381	0.0337	0.0431	0.0608
	Recall@32	0.0140	0.0135	0.0143	0.0139	0.0137	0.0123	0.0184	0.0248
Interactive Rec	Pass@1	–	–	–	–	–	–	0.0660	0.0650
	Pass@32	–	–	–	–	–	–	0.3170	0.3340
	Recall@32	–	–	–	–	–	–	0.1941	0.2051
Label Pred.	AUC	0.6244	0.6598	0.6640	0.6581	0.6204	0.6675	0.6184	0.6821
Item Understand.	LLM-Judge Score	–	–	–	–	–	–	0.3175	0.3984
Rec. Explanation	LLM-Judge Score	–	–	–	–	–	–	3.3540	3.8872

the planner, executor, and compiler turn each request’s flat history into a context card, and the base model is lightly LoRA-fine-tuned to read it. Decoding and evaluation are kept identical across conditions. The comparison therefore measures the effect of adding the AREC layer together with the lightweight adaptation needed for the base generator to read the enriched prompt.

4.2 Implementation Details

The four offline stores are built once from the public training split: the profile and label-behavior stores key on user ID, the collaborative store realizes Eq. (3) (frequency-filtered and stored sparsely), and the item-text store is a BM25 index over captions. The context-card budget is $B = 1024$ tokens. Fine-tuning uses LoRA on the standard projection matrices in bf16 with gradient checkpointing, optimizing Eq. (5) on the answer span only; training data are RecIF samples reconstructed from the release split—disjoint from the test users, which the benchmark forms by holding out 20% of users entirely [22], and which we re-verify with an explicit user-ID leakage check—mixed with a small fraction of general-instruction data. Context cards can be generated on the fly by the planner-executor pipeline or loaded from a pre-computed cache. In the reported setting the base generator is adapted with LoRA while the underlying OneRec architecture and SID vocabulary remain unchanged.

4.3 Evaluation Metrics and Baselines

Following the official RecIF-Bench protocol, recall-style tasks use constrained beam search over the three SID positions and report **Pass@1**, **Pass@32**, and **Recall@32**; the binary task is scored by **AUC** over the next-token probability of the yes/no answer; and the two free-form tasks by an **LLM-judge score** [22]. We compare

against the leaderboard baselines at a comparable scale, spanning classical sequential recommenders (SASRec, BERT4Rec, GRU4Rec), recent scaling-oriented architectures (HSTU [20], ReaRec [14]), and SID-based generative recommenders (TIGER [11], ONEREC-1.7B [22]); AREC applies our enrichment layer on top of ONEREC-1.7B. Since AREC is a model-preserving layer that fine-tunes the 1.7B base, our comparison centers on this 1.7B core rather than on larger backbones, isolating the contribution of enrichment from that of raw model scale. Several baselines do not support the interactive and free-form tasks and are marked “–”.

4.4 Results and Analysis

Table 1 reports all eight RecIF tasks. Read against the un-enriched ONEREC-1.7B column, AREC improves the main outcomes across the three task families, while the only exception is a marginal Pass@1 change on interactive recommendation. Since decoding and evaluation are kept fixed, the comparison isolates the effect of applying AREC together with the lightweight adaptation required for the generator to read enriched prompts.

4.4.1 Structured evidence drives recall-style gains. The gains concentrate exactly where the relevant typed evidence is most informative and least recoverable from a flat history. The cross-domain tasks benefit most: a single-domain transcript hides the very signal these tasks depend on—how content engagement transfers into commercial intent—so making that transfer explicit removes a bottleneck the base model would otherwise have to overcome implicitly on every forward pass. Label-conditioned recommendation benefits for a related reason: the typed label slices, foregrounded by the CONDITIONS edge, supply the behavioral constraint the task is defined

by, which a position-agnostic history dilutes. Short-video recommendation, already the base model’s strongest setting, improves more modestly, since profile and recent-intent evidence largely re-expresses information the long single-domain history already carries; the value added there is sharper prioritization rather than new signal. The pattern is consistent: enrichment helps in proportion to the gap between what the task requires and what a flat history makes legible.

4.4.2 Binary and free-form tasks also benefit. The gains are not limited to SID recall. Label prediction improves because a pointwise yes/no decision benefits from an explicit summary of user taste and recent intent, which a flat history only leaves implicit. The two free-form tasks also improve: item understanding and recommendation explanation both receive cleaner user- and item-level evidence through the card. This result is important for AREC’s positioning. The enrichment layer is designed for recommendation, but it does not merely trade recall for language-side behavior; it strengthens the personalized generation tasks as well.

4.4.3 Efficiency. The enrichment path adds dictionary lookups and sparse-matrix access. The planner and compiler used in our reported setting are deterministic and make no model calls, so no extra LLM call is added to the serving path.

5 Conclusion

We presented AREC, an agentic retrieval-enrichment layer that wraps an unmodified SID-based generative recommender with typed memory, typed tools, a planner, an executor that builds a typed evidence graph, and a compiler that serializes that graph into a faithful, budget-bounded context card. The results point to a simple lesson: for generative recommendation, useful context is not merely more context. It is context organized around the kinds of evidence the task needs and expressed in a form the SID decoder can already read. On RecIF-Bench, AREC improves recall-style recommendation and also strengthens binary and free-form personalized tasks. Future work can study richer typed memories and stronger criteria for when enrichment should be applied.

Acknowledgement

This research was supported in part by Guangdong Provincial Key Field R&D (Grant No. 2024B0101040008), in part by Guangdong Provincial Science and Technology Program (Grant No. 2025A0505080003), and in part by Guangzhou Science and Technology Plan Project - Key R&D Plan (Grant No. 2024B01W0007).

References

- [1] Keqin Bao, Jizhi Zhang, Yang Zhang, Wenjie Wang, Fuli Feng, and Xiangnan He. 2023. Tallrec: An effective and efficient tuning framework to align large language model with recommendation. In *Proceedings of the 17th ACM conference on recommender systems*. 1007–1014.
- [2] Jiaxin Deng, Shiyao Wang, Kuo Cai, Lejian Ren, Qigen Hu, Weifeng Ding, Qiang Luo, and Guorui Zhou. 2025. Onerec: Unifying retrieve and rank with generative recommender and iterative preference alignment. *arXiv preprint arXiv:2502.18965* (2025).
- [3] Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. 2022. Recommendation as language processing (rlp): A unified pretrain, personalized prompt & predict paradigm (p5). In *Proceedings of the 16th ACM conference on recommender systems*. 299–315.
- [4] Yupeng Hou, Zhankui He, Julian McAuley, and Wayne Xin Zhao. 2023. Learning vector-quantized item representation for transferable sequential recommenders. In *Proceedings of the ACM Web Conference 2023*. 1162–1171.
- [5] Clark Mingxuan Ju, Liam Collins, Leonardo Neves, Bhuvish Kumar, Louis Yufeng Wang, Tong Zhao, and Neil Shah. 2025. Generative Recommendation with Semantic IDs: A Practitioner’s Handbook. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 6420–6425.
- [6] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.
- [7] Jiayi Liao, Sihang Li, Zhengyi Yang, Jiancan Wu, Yancheng Yuan, Xiang Wang, and Xiangnan He. 2024. Llara: Large language-recommendation assistant. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1785–1795.
- [8] Zhanyu Liu, Shiyao Wang, Xingmei Wang, Rongzhou Zhang, Jiaxin Deng, Honghui Bao, Jinghao Zhang, Wuchao Li, Pengfei Zheng, Xiangyu Wu, et al. 2025. Onerec-think: In-text reasoning for generative recommendation. *arXiv preprint arXiv:2510.11639* (2025).
- [9] Reza Yousefi Maragheh and Yashar Deldjoo. 2025. The future is agentic: Definitions, perspectives, and open challenges of multi-agent recommender systems. *arXiv preprint arXiv:2507.02097* (2025).
- [10] Filip Radlinski, Krisztian Balog, Fernando Diaz, Lucas Dixon, and Ben Wedin. 2022. On natural language user profiles for transparent and scrutable recommendation. In *Proceedings of the 45th international ACM SIGIR conference on research and development in information retrieval*. 2863–2874.
- [11] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Hulikal Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Tran, Jonah Samost, et al. 2023. Recommender systems with generative retrieval. *Advances in Neural Information Processing Systems* 36 (2023), 10299–10315.
- [12] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems* 36 (2023), 68539–68551.
- [13] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems* 36 (2023), 8634–8652.
- [14] Jiakai Tang, Sunhao Dai, Teng Shi, Jun Xu, Xu Chen, Wen Chen, Jian Wu, and Yuning Jiang. 2026. Think before recommend: Unleashing the latent reasoning power for sequential recommendation. *IEEE Transactions on Knowledge and Data Engineering* (2026).
- [15] Wenjie Wang, Honghui Bao, Xinyu Lin, Jizhi Zhang, Yongqi Li, Fuli Feng, See-Kiong Ng, and Tat-Seng Chua. 2024. Learnable item tokenization for generative recommendation. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 2400–2409.
- [16] Junda Wu, Cheng-Chun Chang, Tong Yu, Zhankui He, Jianing Wang, Yupeng Hou, and Julian McAuley. 2024. Coral: Collaborative retrieval-augmented large language models improve long-tail recommendation. In *Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining*. 3391–3401.
- [17] Likang Wu, Zhi Zheng, Zhaopeng Qiu, Hao Wang, Hongchao Gu, Tingjia Shen, Chuan Qin, Chen Zhu, Hengshu Zhu, Qi Liu, et al. 2024. A survey on large language models for recommendation. *World Wide Web* 27, 5 (2024), 60.
- [18] Yunjia Xi, Weiwen Liu, Jianghao Lin, Xiaoling Cai, Hong Zhu, Jieming Zhu, Bo Chen, Ruiming Tang, Weinan Zhang, and Yong Yu. 2024. Towards open-world recommendation with knowledge augmentation from large language models. In *Proceedings of the 18th ACM Conference on Recommender Systems*. 12–22.
- [19] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629* (2022).
- [20] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Michael He, et al. 2024. Actions speak louder than words: Trillion-parameter sequential transducers for generative recommendations. *arXiv preprint arXiv:2402.17152* (2024).
- [21] Bowen Zheng, Yupeng Hou, Hongyu Lu, Yu Chen, Wayne Xin Zhao, Ming Chen, and Ji-Rong Wen. 2024. Adapting large language models by integrating collaborative semantics for recommendation. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 1435–1448.
- [22] Guorui Zhou, Honghui Bao, Jiaming Huang, Jiaxin Deng, Jinghao Zhang, Junda She, Kuo Cai, Lejian Ren, Lu Ren, Qiang Luo, et al. 2025. OpenOneRec Technical Report. *arXiv preprint arXiv:2512.24762* (2025).
- [23] Guorui Zhou, Jiaxin Deng, Jinghao Zhang, Kuo Cai, Lejian Ren, Qiang Luo, Qianqian Wang, Qigen Hu, Rui Huang, Shiyao Wang, et al. 2025. Onerec technical report. *arXiv preprint arXiv:2506.13695* (2025).
- [24] Joyce Zhou, Yijia Dai, and Thorsten Joachims. 2024. Language-based user profiles for recommendation. *arXiv preprint arXiv:2402.15623* (2024).

A Context Card Example

A representative natural-language card for a short-video request is shown below, with SIDs abbreviated as `<|sid|>` for readability.

[long-term] This user’s long-term profile shows steady engagement with video content (502 interactions, long-view rate 28.9%); preferred items include `<|sid|>` `<|sid|>` `<|sid|>`.
[labels] Items the user actively liked include `<|sid|>` `<|sid|>`; no not-interested signals recorded.
[recent] In the most recent interactions the user engaged with `<|sid|>` `<|sid|>`.
[neighbors] Collaborative filtering identifies co-engaged items `<|sid|>` `<|sid|>`, suggesting alignment with current interests.

The card is budgeted by evidence type; lower-priority sections are dropped as complete units, and every SID token remains byte-identical to the base model’s vocabulary.

B Planner Tool Mapping

Table 2 summarizes the static task→tool mapping $\pi(\tau)$ used by the rule-based planner in the reported setting. A $\checkmark$ marks a tool invoked for the task. Availability checks may still skip a marked tool when the corresponding store has no usable entry for the user.

Table 2: Task-conditioned tool selection $\pi(\tau)$. P denotes Profile, R denotes RecentIntent, L denotes LabelBehavior, X denotes CrossDomain, and C denotes Collaborative.

Task type	P	R	L	X	C
Short Video Rec	$\checkmark$	$\checkmark$			$\checkmark$
Ad Rec	$\checkmark$	$\checkmark$		$\checkmark$	$\checkmark$
Product Rec	$\checkmark$	$\checkmark$		$\checkmark$	$\checkmark$
Label-Cond. Rec	$\checkmark$	$\checkmark$	$\checkmark$		$\checkmark$
Label Pred.	$\checkmark$	$\checkmark$	$\checkmark$		

C Agent Instruction Prompts

This appendix reports the system instructions used by the LLM-assisted planner and compiler variants implemented in our code-base. These prompts are used when the planner or compiler is instantiated with an LLM backend; the reported serving setting in the main paper uses the deterministic planner-executor-compiler path and therefore does not add an extra LLM call at inference time.

C.1 Planner Agent System Prompt

You are a tool planner for a recommendation system. Your job is to select retrieval tools according to the task type and available data.

Available tools. `profile` builds a long-term user profile from historical interactions and is useful as a foundation. `recent_intent` extracts recent interaction patterns when current interests matter. `label_behavior` analyzes labeled interactions such as like, follow, and longview when label information is available. `cross_domain` finds transfer signals for ad or product recommendation. `collaborative` retrieves items from similar users when the history is sufficient and may reference the recent-intent output.

Task types. The supported task types include video recommendation, ad recommendation, product recommendation, label-conditioned recommendation, label prediction, interactive recommendation, recommendation reasoning, and item understanding. Given a task description, output a JSON object with `tool_calls`, an ordered list of tools to call, and `rationale`, a brief explanation of the tool selection strategy.

Rules. Start with `profile` for most user-centric tasks. Include `recent_intent` when current interests matter. Use `label_behavior` for label-conditioned or label-prediction tasks when labeled interactions are available. Use `cross_domain` for ad and product tasks. Add `collaborative` when the history contains enough interactions to support neighbor retrieval. The collaborative tool may depend on recent intent through a deferred reference such as `$recent_intent.recent_sids[:3]`. Avoid tools that do not help the task.

C.2 Compiler Agent System Prompt

You are a context-card compiler for a generative recommendation system. Your job is to synthesize tool results into a compact natural-language context card that helps the downstream SID-based recommender predict the next item.

The input contains typed tool results, including the tool name, node type, confidence score, estimated token cost, and tool-specific data. Return a JSON object with three fields: `card`, the compiled context card; `kept_signals`, the list of included tool signals; and `dropped`, the list of excluded signals with brief reasons.

The card should read as one coherent narrative rather than a list of disconnected facts. Include concrete SID sequences when available and preserve every SID token exactly. Do not rewrite, split, paraphrase, or partially copy a SID sequence. Be concise and avoid filler. Drop signals whose confidence is too low. For label-conditioned and label-prediction tasks, prioritize label-behavior signals. For ad and product tasks, make reliable cross-domain evidence explicit.

Your goal is to create a compact context card that helps the downstream model use profile, recent intent, label behavior, cross-domain evidence, and collaborative neighbors while staying within the context budget.